PowerShell: from first steps to a complete project

Original learning guide | 14 October 2026 | Read, run, change, explain, practice

All commands assume a PowerShell terminal at the repository root. Code is included for study; use the source files for copying and editing.

10 lessons, ordered from easy to hard. Read PowerShell-Learning-Guide.pdf for the complete printable course, or use the individual lesson files below.

Learn the shell alongside C#. The core scripts use syntax compatible with Windows PowerShell 5.1 and PowerShell 7. Windows PowerShell 5.1 runs on .NET Framework; installing the .NET 10 SDK does not upgrade that shell. PowerShell 7 runs as pwsh and is the recommended modern shell. Optional installation: winget install --id Microsoft.PowerShell --source winget. Open a new terminal and run pwsh afterward.

Before lesson 01

1. Open a PowerShell terminal at the repository root, the folder containing LearningPack.slnx.
2. If local scripts are blocked, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` for this terminal. This changes only this process, not the machine policy.
3. Run `. .\Initialize-Training.ps1` (there is a space between the first and second dots).
4. Keep the terminal at the repository root when copying commands from lessons.
5. For every lesson: predict the output, run it, read each statement, change one thing, complete the exercise, then check SOLUTION.md.

Lesson order

1. 01-Commands: Commands, help, and discovery
2. 02-Variables: Variables, strings, and collections
3. 03-ControlFlow: Conditions and loops
4. 04-Objects: Objects and properties
5. 05-Pipeline: Filtering, sorting, and measuring
6. 06-Functions: Functions and pipeline parameters
7. 07-Files: CSV, JSON, paths, and WhatIf
8. 08-Errors: Errors and external process exit codes
9. 09-Modules: Reusable modules and reporting
10. 10-Capstone: Control the .NET task API

How to know you are ready to move on

Explain the example without reading the notes. Complete the exercise without copying the solution. Make one valid change and one invalid-input experiment. Record what happened in Progress.md. If you cannot explain a result, repeat the lesson with smaller values.

Learn Git with this track

Read the Git workflow and this track's Git checkpoints. Each lesson includes a branch name, verification step, staging paths, and suggested commit. The printable course guide includes the shared Git workflow too.

Official PDF references

The Official-References index lists downloaded publisher PDFs and printable copies of current Microsoft documentation, with source URLs, editions, and course mappings. Publisher PDFs retain their original content. Documentation exports are clearly labeled as local print copies, rather than Microsoft-published PDF books.

Reference documentation

The lesson guides are original teaching material. Official downloads and documentation exports are listed in Official-References/README.md. The links below provide further online study. Checked on 14 October 2026.

- PowerShell documentation
- Install PowerShell on Windows
- Pipeline concepts
- Advanced functions

Course code style

Authored C# follows Google's C# guide, linked from the Google style-guide index.

C# and .NET lessons

- Use two spaces, same-line opening braces, and a 100-column limit.
- Put one statement on each line and brace control-flow bodies.
- Use PascalCase for types/public members, camelCase for parameters/locals, and _camelCase for private fields.
- Place System imports first and sort other imports alphabetically.
- Use block-bodied methods; reserve expression bodies for suitable properties and simple lambdas.
- Put each core type in its own matching source file.
- Use constants for fixed rules and readonly fields for stable dependencies.

.editorconfig records formatter settings and private-field naming. Types now live in files such as StudySession.cs, StudyEntry.cs, and TaskRules.cs. Numbered folders preserve lesson order.

The course still teaches modern C#: top-level programs, file-scoped namespaces, records, collection expressions, and C# 14 field-backed properties. Positional record parameters use PascalCase because they declare public properties.

PowerShell convention

Google's shell guide targets Bash, so these scripts use a documented course convention: two spaces, same-line braces, explicit blocks, and separate statements. Aim for 100 columns; long literal URLs, generated HTML here-strings, and indivisible arguments can exceed that target.

Keep Verb-Noun function names, PascalCase parameters/output properties, and camelCase locals. Use full commands, structured output, help comments, validation, native exit-code checks, and splatting for long calls.

Verify before committing

```
. .\Initialize-Training.ps1
dotnet build .\LearningPack.slnx -m:1 -nr:false
.\Check-Style.ps1
.\Test-LearningPack.ps1
```

To apply the SDK formatter:

```
dotnet format whitespace .\LearningPack.slnx --no-restore
```

```
dotnet format style .\LearningPack.slnx --no-restore --diagnostics IDE0011 IDE0022 IDE0061 --severity
info
```

Review changes and run checks before regenerating course PDFs with Build-Guides.ps1. Original publisher PDFs and official documentation exports retain their source formatting.

Git practice in the PowerShell course

Read Git-Workflow.md first and save the supplied baseline before editing. Start each exercise on a clean branch from main. Run focused checks, stage related changes, commit with a clear message, then choose either local integration or a GitHub pull request.

Track checkpoints

Lesson	Branch	Suggested commit	Extra Git skill
01-Commands	feat/powershell-01-discovery	docs(powershell): record command discovery	Inspect status, diff, and staging
02-Variables	feat/powershell-02-remaining-minutes	feat(powershell): calculate remaining study minutes	Inspect status, diff, and staging
03-ControlFlow	feat/powershell-03-session-count	feat(powershell): count long study sessions	Inspect status, diff, and staging
04-Objects	feat/powershell-04-json-output	feat(powershell): serialize study objects	Keep commits focused and review the staged snapshot
05-Pipeline	feat/powershell-05-short-sessions	feat(powershell): count filtered short sessions	Keep commits focused and review the staged snapshot
06-Functions	feat/powershell-06-study-days	feat(powershell): convert minutes to study days	Keep commits focused and review the staged snapshot
07-Files	feat/powershell-07-json-filter	feat(powershell): filter saved JSON study data	Push a branch and explain validation in a PR
08-Errors	feat/powershell-08-native-exit	test(powershell): check native failure exit codes	Push a branch and explain validation in a PR
09-Modules	feat/powershell-09-average-minutes	feat(powershell): add average minutes to summaries	Push a branch and explain validation in a PR
10-Capstone	feat/powershell-10-task-client	test(powershell): verify task client completion	Review integration history and practice conflict resolution

For a docs-only change, prefer a docs/ branch; for a bug fix, prefer fix/. The suggested feat/ name is a convenient starting point and can be adapted to what you actually implement. Do not describe a documentation-only experiment as a new program feature.

Practice in a separate local repository

```
.\New-GitPractice.ps1 -Topic PowerShell
```

Use the printed developer and teammate paths. Both clones and their bare remote are inside .artifacts. Follow the push, merge, and conflict examples from Git-Workflow.md there. This is separate from changing your actual course repository.

A complete checkpoint

1. Verify the change with the lesson commands and an appropriate boundary case.
2. Inspect git diff and stage only related source or notes.
3. Inspect git diff --cached and commit.
4. Push your branch if you intend to publish it.
5. Review a PR or perform a local merge, then verify the behavior on main.
6. Record the resulting commit ID in Progress.md.

Do not commit generated build output, demonstration data, API logs, browser profiles, or credentials. Regenerate the original course PDFs after documentation changes only when the source content is ready; inspect PDF changes by size and rendered pages rather than a text diff.

Git workflow for all three courses

Use Git while completing exercises. Git records versions locally; GitHub hosts a remote repository and adds pull requests. A branch lets you develop a change before integrating it into main. A commit stores the staged snapshot with a message. Push sends commits to a remote. Merge combines histories.

This course uses short-lived branches, small commits, and reviewed merges. It is one practical workflow; team rules can differ. The optional type(scope): summary message convention is a course recommendation, not a Git requirement.

1. Inspect and set your identity

```
git --version
git status --short --branch
git branch --show-current
git remote -v
git log --oneline --decorate -5
```

The working tree is your files on disk. The index, or staging area, holds the next proposed snapshot. HEAD identifies the current commit. origin is a conventional remote name. This repository already has origin: check its actual address instead of adding another remote. Do not run git init inside the lesson folders; all courses belong to one repository.

If identity is missing, replace these placeholders with your own name and verified or GitHub noreply email:

```
git config --local user.name "Your Real Name"
git config --local user.email "your-verified-or-GitHub-noreply-email"
git config --get user.name
git config --get user.email
```

Local identity settings affect only this repository. Commit identity differs from push authentication. Use Git Credential Manager or GitHub's supported authentication; keep passwords and tokens out of source files and remote URLs.

2. Commit the supplied baseline before exercises

An untracked file does not exist in Git history. If the learning pack is initially uncommitted, save its supplied state before editing lessons:

```
git status --short --branch
git switch -c course/learning-pack
git add -- .gitignore .gitattributes .editorconfig README.md STYLE.md Directory.Build.props global.json
NuGet.Config LearningPack.slnx
git add -- Initialize-Training.ps1 Test-LearningPack.ps1 Check-Style.ps1 Build-Guides.ps1
```

```
git add -- Git-Workflow.md New-GitPractice.ps1 Official-Resources.json Download-OfficialReferences.ps1
git add -- CSharp PowerShell DotNet10 .github
git diff --cached --stat
git diff --cached --check
git commit -m "chore(course): add CSharp PowerShell and .NET 10 learning pack"
```

Review every staged path, including deletions. Confirm bin, obj, output, .tools and .artifacts are excluded. Inspect PDF sizes. Add only paths that exist in your copy.

Choose ONE baseline integration path:

- Local practice: git switch main, then git merge --no-ff course/learning-pack -m "Merge initial learning pack".
- GitHub practice: git push -u origin course/learning-pack, open a pull request to main, merge on GitHub, then fetch and fast-forward your local main.

Do not merge locally and also squash the same change on GitHub: those are different histories. If the baseline is already committed on main, skip this section.

3. Start a focused branch

Start with a clean working tree. Commit your current work or deliberately stash it before switching.

```
git switch main
git status --short
# For GitHub practice; omit fetch/pull in local-only practice:
git fetch origin
git pull --ff-only origin main
git switch -c feat/csharp-01-city
```

--ff-only refuses to combine divergent local and remote histories silently. If it refuses, inspect the graph and follow the synchronization section. Names such as feat/csharp-01-city, fix/powershell-09-minute-validation, and feat/dotnet10-task-summary identify one purpose. Branches are repository-wide, not restricted to the named folder. Integrate a lesson branch before starting the next unless you intentionally need a dependent branch.

4. Edit, verify, stage, and commit

For the first C# exercise, implement the optional city argument and verify both explicit and default inputs:

```
. .\Initialize-Training.ps1
dotnet run --project .\CSharp\01-Hello -- Dejan Ljubljana
dotnet run --project .\CSharp\01-Hello -- Dejan
git diff -- CSharp/01-Hello/Program.cs
git add -- CSharp/01-Hello/Program.cs
git diff --cached -- CSharp/01-Hello/Program.cs
git diff --cached --check
git status --short
```

```
git commit -m "feat(csharp): accept an optional city argument" -m "Use Ljubljana when the city is
omitted. Verified explicit and default inputs."
git show --stat HEAD
```

diff shows unstaged edits. diff --cached shows exactly what a commit will contain. Untracked files need editor inspection and staging before they appear in the staged diff. If you edit after staging, add again for edits you want in the same snapshot. git add -p stages selected hunks from tracked files.

Keep each commit focused and working. Use the lesson's verification commands. The root test script checks the supplied behavior; deliberate exercise changes may require updating its expectations.

5. Write clear messages and PR comments

Use an imperative summary describing the resulting change. A roughly 50-72-character summary is a useful readability target, not a Git limit. A body explains why a non-obvious decision was made and what relevant checks passed. Each additional -m adds another paragraph.

Type	Purpose	Example
feat	New behavior	feat(csharp): accept an optional city argument
fix	Correct behavior	fix(powershell): reject negative study minutes
docs	Documentation	docs(dotnet10): explain API validation responses
refactor	Structure with preserved behavior	refactor(csharp): extract study calculations
test	Verification	test(dotnet10): check task persistence
chore	Maintenance	chore(course): normalize source line endings

This is an optional convention. Avoid vague messages such as updates, fix stuff, or final. Source comments explain implementation details; commit messages explain a recorded change; PR comments discuss a proposed change and review. They have different purposes.

For a PR, state the problem, resulting behavior, and exact validation. The supplied .github/PULL_REQUEST_TEMPLATE.md provides those fields. Example title: Accept an optional city in the greeting. Example description: The greeting accepts a second argument and defaults to Ljubljana. Verified explicit and omitted city runs.

6. Push and review a pull request

```
git remote -v
git push -u origin feat/csharp-01-city
```

-u records the upstream; subsequent git push uses that relationship. Push publishes commits but does not merge main. On GitHub, create a pull request with main as base and your feature branch as head. Review Files changed, fix failed checks, and respond to comments. Additional pushes to that branch update the PR.

If a push is rejected, read the error: authentication, branch protection, and remote changes require different fixes. Fetch and inspect before attempting any history rewrite.

7. Merge: choose one completion path

Method	History effect	Useful context
Fast-forward	Moves the target pointer to a descendant	Simple local exercises
Merge commit	Records two parent histories	Preserve individual commits and integration point
Squash merge	Creates one new combined commit	One compact reviewed feature
Rebase then merge	Replays commits on a new base	Teams requiring linear history

Use a merge commit in the local lab to see branch history. For a small GitHub PR, squash merge is a reasonable default if repository policy allows it. Review the final squash message because it becomes main's commit message.

Local completion, without publishing:

```
git switch main
git status --short
git merge --no-ff feat/csharp-01-city -m "Merge CSharp city argument exercise"
git log --oneline --graph --decorate -10
git branch -d feat/csharp-01-city
```

Merge changes your CURRENT branch. --no-ff requests an explicit merge commit. Delete a branch only after integration. -d checks ancestry. After a GitHub squash merge, -d may refuse because the original commits were not merged by identity; keeping the branch is fine.

GitHub completion: merge the PR on GitHub first, then:

```
git fetch origin
git switch main
git pull --ff-only origin main
git log --oneline --graph --decorate -10
git status --short
```

Check the completed behavior is on main before starting the next lesson. git fetch --prune removes stale remote-tracking names, not your local branches.

To incorporate updated main into an active feature branch:

```
git switch feat/csharp-01-city
git fetch origin
git merge origin/main
# Resolve conflicts if needed, verify, and finish the merge.
git push
```

Rebase is another choice for a private, never-pushed branch. It changes commit IDs. This course uses merge to update published branches without rewriting their shared history.

8. Resolve conflicts and undo deliberately

A conflict means overlapping changes need your decision. Run status, open the file, find <<<<<<<, =======, and >>>>>>>, write the intended combined result, remove the markers, and verify it. Stage each resolved file and commit:

```
git status
# Edit and run checks BEFORE staging:
git add -- CSharp/01-Hello/Program.cs
git diff --cached
git commit -m "merge(csharp): preserve city input and updated greeting"
```

git merge --abort abandons the merge where possible. Start merges with a clean working tree so the pre-merge state is clear. Practice your first conflict in the isolated lab.

Command	Effect	Appropriate use
git restore --staged -- path	Unstage while keeping working edits	Wrong path staged
git restore -- path	Discard unstaged tracked-file edits	Only after reviewing and choosing to discard
git stash push -u -m "lesson draft"	Park edits and untracked files locally	Switch context temporarily
git stash list	Show parked work	Find your draft
git stash apply	Restore the latest stash while keeping it	Resume, inspect, then drop after verification
git revert commit-id	Record the inverse of a normal commit	Undo shared work without rewriting history

Stash is local temporary storage, not a pushed backup. Reverting a merge requires additional parent selection; the simple revert example is for a normal commit. Amend rewrites the latest commit; reserve it for appropriate private, unpushed work. Routine beginner work does not require reset --hard, clean -fd, or force pushes.

9. Practice pushing and conflicts offline

New-GitPractice.ps1 creates a fresh developer repository, bare local remote, and teammate clone under .artifacts. It copies your selected topic's first runnable lesson and configures fictional identities only inside those lab repositories. It does not change the course repository's branch or contact GitHub.

```
.\New-GitPractice.ps1 -Topic CSharp
# Use the developer path printed by the script:
Set-Location 'PASTE-THE-PRINTED-DEVELOPER-FOLDER'
git switch -c feat/practice-note
Add-Content -LiteralPath PracticeNotes.txt -Value 'I can explain the staging area.'
git diff
git add -- PracticeNotes.txt
git diff --cached
git commit -m "docs(csharp): record staging practice"
git push -u origin feat/practice-note
```

```
git switch main
git merge --no-ff feat/practice-note -m "Merge staging practice"
git push origin main
git log --oneline --graph --all
```

Open another terminal in the printed teammate folder. Run git pull --ff-only origin main and inspect PracticeNotes.txt. This demonstrates the same data movement without publishing to your GitHub account.

In the lab developer folder, create a deliberate conflict:

```
git switch main
git switch -c feat/conflict-note
Set-Content -LiteralPath PracticeNotes.txt -Value 'Feature: validate inputs before saving.'
git add -- PracticeNotes.txt
git commit -m "docs(course): describe validation"
git switch main
Set-Content -LiteralPath PracticeNotes.txt -Value 'Main: verify persistence after saving.'
git add -- PracticeNotes.txt
git commit -m "docs(course): describe persistence"
git merge feat/conflict-note
# EXPECT a conflict and nonzero exit code. Resolve the file:
git status
Get-Content -LiteralPath PracticeNotes.txt
Set-Content -LiteralPath PracticeNotes.txt -Value 'Validate inputs before saving, then verify
persistence.'
git add -- PracticeNotes.txt
git commit -m "merge(course): combine validation and persistence notes"
git log --oneline --graph --all
```

Both branches replaced the same text differently. Your resolution preserves both ideas. Return to the original repository root when finished.

10. Apply Git in every course

Each lesson contains a Git checkpoint with a branch name, focused verification, staging paths, and suggested message. Git-Practice.md maps your track. Use lessons 01-03 for status, branches, and staging; middle lessons for focused commits and review; final lessons for PRs, integration, and conflict practice.

For a combined API/client change, one focused branch can include both DotNet10 and PowerShell files because they implement one contract. Include only related changes. Keep generated files and secrets out of history.

Official references

This is original instructional material. Further detail comes from Git's command manuals and GitHub's workflow guide:

- git status
- git add

- git commit
- git merge
- git push
- git fetch
- git rebase
- GitHub flow

PowerShell: official reference PDFs

This folder contains **5 PDFs** from official source material. Files labeled Publisher PDF are the original downloaded publications. Files labeled Documentation export are locally printed copies of official Microsoft Learn articles, with source links, attribution, retrieval dates, and print formatting. They are not Microsoft-published PDF books or complete product manuals.

Read in this order

Use the course's original learning guide and runnable lessons first. Then consult the relevant current Microsoft documentation article for a deeper explanation. Read standards and architecture books when you want formal language/runtime details or further architecture study.

The Monad Manifesto is the original 2002 design paper. It explains the ideas behind PowerShell, not current command syntax. The documentation exports use the PowerShell 7.6 view; the course's core scripts remain compatible with Windows PowerShell 5.1.

PDF catalog

PDF	Kind	Read with course lessons	Scope / edition
Microsoft-Monad-Manifesto.pdf	Publisher PDF	Background; 01 and 04	Jeffrey Snover's original 2002 PowerShell design paper. Historical background, not a current command reference.
Microsoft-PowerShell-Overview.pdf	Documentation export	01	Current Microsoft Learn PowerShell overview, view 7.6.
Microsoft-PowerShell-Pipelines.pdf	Documentation export	04-05	Microsoft Learn object pipeline reference, view 7.6.
Microsoft-PowerShell-Advanced-Functions.pdf	Documentation export	06	Microsoft Learn advanced functions reference, view 7.6.
Microsoft-PowerShell-Modules.pdf	Documentation export	09	Microsoft Learn module reference, view 7.6.

Sources

- Microsoft-Monad-Manifesto.pdf: official source and PDF download.
- Microsoft-PowerShell-Overview.pdf: official source.
- Microsoft-PowerShell-Pipelines.pdf: official source.
- Microsoft-PowerShell-Advanced-Functions.pdf: official source.
- Microsoft-PowerShell-Modules.pdf: official source.

Provenance and refresh

Download-Record.json records the source URL, retrieval timestamp in UTC, file size, PDF SHA-256 hash, and original HTML hash for each documentation export. The retrieval timestamp records when the copy was collected; it is not the document's publication date. Article metadata dates are preserved where supplied. The official article remains the source of truth for changes after this snapshot.

From the repository root:

```
.\Download-OfficialReferences.ps1
```

This refreshes the collection using Official-Resources.json. It requires internet access and local Chrome or Edge. It downloads only the public documents and creates local print copies; it does not install software or publish anything.

Attribution

Original publisher PDFs retain their own copyright, license notices, and source formatting. Ecma standards include their copying terms. Microsoft Learn article wording and examples are retained while navigation is removed and print styling and a provenance banner are added. Credit belongs to Microsoft and the documentation contributors, including named authors credited in the source. See the Microsoft Learn terms and the .NET documentation CC BY 4.0 license. This course and its exports do not imply publisher endorsement.

01-Commands: Commands, help, and discovery

Level: Beginner | Suggested time: 30-45 minutes

Goal

Learn finding commands and understanding the shell. Complete the setup in the track README.

Understand the idea

PowerShell is a shell and a scripting language. A cmdlet is a command named with a verb and noun, such as Get-Date. Parameters begin with a dash. Get-Command discovers a command and its type; Get-Help explains syntax and examples. Help content depends on what is installed locally; you can still inspect command syntax without downloading help. The pipe character passes output to another command. A .ps1 file stores a script. Dot-slash tells PowerShell to run a file from the current directory. Full command names are easier to read than aliases such as gci when you are learning.

Step by step

1. Open Demo.ps1 in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
.\PowerShell\01-Commands\Demo.ps1
```

Expected result

```
The Get-Date command, today's local date, and help information. The date and installed help vary.
```

Exercise

Find Get-Content, inspect its syntax, and list only files in this lesson folder.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Get-ChildItem accepts -LiteralPath and -File.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/powershell-01-discovery.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/powershell-01-discovery
# Implement the exercise, then verify:
.\PowerShell\01-Commands\Demo.ps1
git diff -- PowerShell/01-Commands/Demo.ps1
git add -- PowerShell/01-Commands/Demo.ps1
git diff --cached
git diff --cached --check
git commit -m "docs(powershell): record command discovery"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Explain the difference between working edits and the staged snapshot.

For GitHub practice, publish deliberately with `git push -u origin feat/powershell-01-discovery`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

Demo.ps1

```
[CmdletBinding()]
param()
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

# Verb-Noun names describe actions. Inspect before automating.
Get-Command Get-Date | Select-Object Name, CommandType
Get-Date -Format 'yyyy-MM-dd'
Get-Help Get-ChildItem -Examples
Write-Output 'Tip: use Get-Command and Get-Help whenever a command is unfamiliar.'
```

01-Commands exercise solution

Find Get-Content, inspect its syntax, and list only files in this lesson folder.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
Get-Command Get-Content -Syntax
Get-ChildItem -LiteralPath $PSScriptRoot -File
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

02-Variables: Variables, strings, and collections

Level: Beginner | Suggested time: 30-45 minutes

Goal

Learn typed values, interpolation, arrays, and hashtables. Complete 01-Commands first.

Understand the idea

Variable names start with a dollar sign. PowerShell can infer a type, or a declaration such as [int] can constrain it. Double-quoted strings expand variables; single-quoted strings preserve literal text. Use $() to evaluate an expression inside an expandable string, such as a property access. @() constructs an array. @{} constructs a hashtable of keys and values. The -f operator substitutes numbered placeholders into a format string. Hashtable member syntax is convenient for simple named keys. Avoid names used by automatic variables, including HOME and PID. PowerShell arrays also use zero-based indexing.

Step by step

1. Open Demo.ps1 in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
.\PowerShell\02-Variables\Demo.ps1 -Name Dejan
```

Expected result

```
Hello, Dejan!
Topic: PowerShell; minutes: 30
Topic count: 3
Single quotes keep $minutes as literal text.
```

Exercise

Add a Goal field of 60 minutes to the hashtable and print the remaining minutes.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Subtract the current Minutes value from Goal.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/powershell-02-remaining-minutes.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/powershell-02-remaining-minutes
# Implement the exercise, then verify:
.\PowerShell\02-Variables\Demo.ps1
git diff -- PowerShell/02-Variables/Demo.ps1
git add -- PowerShell/02-Variables/Demo.ps1
git diff --cached
git diff --cached --check
git commit -m "feat(powershell): calculate remaining study minutes"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Explain the difference between working edits and the staged snapshot.

For GitHub practice, publish deliberately with `git push -u origin feat/powershell-02-remaining-minutes`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

Demo.ps1

```
[CmdletBinding()]
param([string]$Name = 'Learner')
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

[int]$minutes = 30
$topics = @('C#', 'PowerShell', '.NET 10')
$plan = @{
  Topic = $topics[1]
  Minutes = $minutes
}
Write-Output "Hello, $Name!"
Write-Output ('Topic: {0}; minutes: {1}' -f $plan.Topic, $plan.Minutes)
Write-Output "Topic count: $($topics.Count)"
Write-Output 'Single quotes keep $minutes as literal text.'
```

02-Variables exercise solution

Add a Goal field of 60 minutes to the hashtable and print the remaining minutes.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
$plan.Goal = 60
Write-Output "Remaining: $($plan.Goal - $plan.Minutes) minutes"
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

03-ControlFlow: Conditions and loops

Level: Beginner | Suggested time: 30-45 minutes

Goal

Learn comparison operators, branching, and repetition. Complete 02-Variables first.

Understand the idea

PowerShell comparisons use named operators: -eq, -ne, -gt, -ge, -lt and -le. Do not use a single equals sign to compare; it assigns a value. if chooses a branch. foreach visits items in a collection. A for loop is useful when you need a counter. ValidateRange rejects a parameter outside its allowed range before the script body runs. PowerShell strings compare case-insensitively by default; -ceq is case-sensitive equality. A while loop repeats while a condition holds and requires a way to finish. Prefer parameter validation over trusting input.

Step by step

1. Open Demo.ps1 in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
.\PowerShell\03-ControlFlow\Demo.ps1 -Minutes 30
```

Expected result

```
Long session
Total: 60 minutes
Round 1
Round 2
Round 3
```

Exercise

Count how many values in 10, 20, 30, 40 are at least 30. Expect 2.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Use a counter, foreach, and -ge.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/powershell-03-session-count.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/powershell-03-session-count
# Implement the exercise, then verify:
.\PowerShell\03-ControlFlow\Demo.ps1
git diff -- PowerShell/03-ControlFlow/Demo.ps1
git add -- PowerShell/03-ControlFlow/Demo.ps1
git diff --cached
git diff --cached --check
git commit -m "feat(powershell): count long study sessions"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Explain the difference between working edits and the staged snapshot.

For GitHub practice, publish deliberately with `git push -u origin feat/powershell-03-session-count`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

Demo.ps1

```
[CmdletBinding()]
param([ValidateRange(1, 180)][int]$Minutes = 30)
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

if ($Minutes -ge 30) {
  Write-Output 'Long session'
} else {
  Write-Output 'Short session'
}

$total = 0
foreach ($duration in @(10, 20, $Minutes)) {
  $total += $duration
}
Write-Output "Total: $total minutes"
for ($round = 1; $round -le 3; $round++) {
  Write-Output "Round $round"
}
```

03-ControlFlow exercise solution

Count how many values in 10, 20, 30, 40 are at least 30. Expect 2.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
$count = 0
foreach ($duration in @(10, 20, 30, 40)) {
  if ($duration -ge 30) {
    $count++
  }
}
Write-Output "Long sessions: $count"
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

04-Objects: Objects and properties

Level: Intermediate | Suggested time: 45-60 minutes

Goal

Learn structured output and object inspection. Complete 03-ControlFlow first.

Understand the idea

PowerShell usually passes objects through its pipeline. An object has properties containing data and methods providing operations. PSCustomObject creates an object with named fields from a hashtable. Get-Member reveals the object's type, properties, and methods. Selecting a property is different from converting an object to text. Select-Object chooses fields while preserving objects. Format-Table and Format-List create display instructions and should appear only at the end of a display pipeline. Keep data structured until the final presentation, export, or serialization step. This is why PowerShell can sort by Minutes without parsing printed columns.

Step by step

1. Open Demo.ps1 in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
.\PowerShell\04-Objects\Demo.ps1
```

Expected result

```
An object with Topic=PowerShell, Minutes=30, Complete=True; member information; Topic property:
PowerShell. Table spacing varies.
```

Exercise

Create a second object and export both to JSON in the console. Confirm it contains property names.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Use an array of objects and ConvertTo-Json.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/powershell-04-json-output.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/powershell-04-json-output
# Implement the exercise, then verify:
.\PowerShell\04-Objects\Demo.ps1
git diff -- PowerShell/04-Objects/Demo.ps1
git add -- PowerShell/04-Objects/Demo.ps1
git diff --cached
git diff --cached --check
git commit -m "feat(powershell): serialize study objects"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Keep this commit limited to one exercise; explain the rule and the verification in its message body.

For GitHub practice, publish deliberately with `git push -u origin feat/powershell-04-json-output`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

Demo.ps1

```
[CmdletBinding()]
param()
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

$session = [pscustomobject]@{
  Topic = 'PowerShell'
  Minutes = 30
  Complete = $false
}
$session.Complete = $true
Write-Output $session
$session | Get-Member | Select-Object -First 6 Name, MemberType
Write-Output "Topic property: $($session.Topic)"
```

04-Objects exercise solution

Create a second object and export both to JSON in the console. Confirm it contains property names.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
$second = [pscustomobject]@{
  Topic = 'C#'
  Minutes = 20
  Complete = $false
}
@($session, $second) | ConvertTo-Json -Depth 3
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

05-Pipeline: Filtering, sorting, and measuring

Level: Intermediate | Suggested time: 45-60 minutes

Goal

Learn pipeline objects, the current item, and aggregation. Complete 04-Objects first.

Understand the idea

A pipeline connects command output to command input. Where-Object filters objects with a script block. $_ is the current pipeline item inside that block. Sort-Object sorts using a property. ForEach-Object applies work to each item; foreach is a language statement used differently. Measure-Object computes aggregates. Assigning pipeline output captures the resulting objects. With zero or one results, ordinary assignment may give null or a single object rather than an array; @(...pipeline...) normalizes the result to an array. Use -LiteralPath when a path should be treated exactly instead of as a wildcard expression.

Step by step

1. Open Demo.ps1 in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
.\PowerShell\05-Pipeline\Demo.ps1
```

Expected result

```
C#: 45 minutes
.NET 10: 30 minutes
Total: 95 minutes
```

Exercise

Filter sessions below 30 minutes and print their count. Then group by Topic.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Wrap the filtered pipeline in @() before reading Count.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/powershell-05-short-sessions.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/powershell-05-short-sessions
# Implement the exercise, then verify:
.\PowerShell\05-Pipeline\Demo.ps1
git diff -- PowerShell/05-Pipeline/Demo.ps1
git add -- PowerShell/05-Pipeline/Demo.ps1
git diff --cached
git diff --cached --check
git commit -m "feat(powershell): count filtered short sessions"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Keep this commit limited to one exercise; explain the rule and the verification in its message body.

For GitHub practice, publish deliberately with `git push -u origin feat/powershell-05-short-sessions`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

Demo.ps1

```
[CmdletBinding()]
param()
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

$sessions = @(
  [pscustomobject]@{
    Topic = 'C#'
    Minutes = 45
  }
  [pscustomobject]@{
    Topic = 'PowerShell'
    Minutes = 20
  }
  [pscustomobject]@{
    Topic = '.NET 10'
    Minutes = 30
  }
)
$sessions | Where-Object {
  $_.Minutes -ge 30
} |
  Sort-Object Minutes -Descending |
  ForEach-Object {
  '{0}: {1} minutes' -f $_.Topic, $_.Minutes
}
$measure = $sessions | Measure-Object -Property Minutes -Sum -Average
Write-Output "Total: $($measure.Sum) minutes"
```

05-Pipeline exercise solution

Filter sessions below 30 minutes and print their count. Then group by Topic.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
$short = @($sessions | Where-Object {
    $_.Minutes -lt 30
})
```

```
Write-Output "Short sessions: $($short.Count)"
$sessions | Group-Object Topic | Select-Object Name, Count
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

06-Functions: Functions and pipeline parameters

Level: Intermediate | Suggested time: 45-60 minutes

Goal

Learn advanced functions, process blocks, and output streams. Complete 05-Pipeline first.

Understand the idea

A function groups reusable commands. CmdletBinding enables common parameters such as -Verbose and -ErrorAction. Mandatory requires the caller to supply a value. ValueFromPipeline binds each input object to the parameter. The process block executes for each pipeline item. ValidateRange rejects unsuitable values. PowerShell emits every uncaptured expression on the success stream, so accidental output inside a function can contaminate its result. Use Write-Verbose for diagnostic messages and structured objects for results. Unlike C#, return is not the only way to produce output; it also ends the current function or block execution path.

Step by step

1. Open Demo.ps1 in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
.\PowerShell\06-Functions\Demo.ps1
```

Expected result

```
30 minutes = 0.5 hours
60 minutes = 1 hours
90 minutes = 1.5 hours
(Decimal separators follow your shell culture.)
```

Exercise

Add a Days property where one study day is eight hours. Verify 480 minutes produces Days=1.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Divide Minutes by 60 and then by 8 in the output object.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/powershell-06-study-days.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/powershell-06-study-days
# Implement the exercise, then verify:
.\PowerShell\06-Functions\Demo.ps1
git diff -- PowerShell/06-Functions/Demo.ps1
git add -- PowerShell/06-Functions/Demo.ps1
git diff --cached
git diff --cached --check
git commit -m "feat(powershell): convert minutes to study days"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Keep this commit limited to one exercise; explain the rule and the verification in its message body.

For GitHub practice, publish deliberately with `git push -u origin feat/powershell-06-study-days`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

Demo.ps1

```
[CmdletBinding()]
param()
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

function ConvertTo-StudyHours {
  <#
  .SYNOPSIS
  Convert each pipeline minute value into a structured hours result.
  #>
  [CmdletBinding()]
  param(
    [Parameter(Mandatory, ValueFromPipeline)]
    [ValidateRange(0, 100000)]
    [double]$Minutes
  )
  process {
    # Return data on the success stream, one object per input.
    [pscustomobject]@{
      Minutes = $Minutes
      Hours = $Minutes / 60
    }
  }
}
30, 60, 90 | ConvertTo-StudyHours |
  ForEach-Object {
  '{0} minutes = {1} hours' -f $_.Minutes, $_.Hours
}
```

06-Functions exercise solution

Add a Days property where one study day is eight hours. Verify 480 minutes produces Days=1.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
[pscustomobject]@{
  Minutes = $Minutes
  Hours = $Minutes / 60
  Days = $Minutes / 480
```

```
}
// Replace the output object inside process.
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

07-Files: CSV, JSON, paths, and WhatIf

Level: Intermediate | Suggested time: 45-60 minutes

Goal

Learn persisting data and previewing changes. Complete 06-Functions first.

Understand the idea

PSScriptRoot is the directory containing the running script, so paths remain stable when the terminal moves. Join-Path joins path components. Export-Csv writes a structured table with column names. Import-Csv returns objects, but CSV field values are strings; cast numeric fields before arithmetic. JSON preserves richer structures and uses a depth limit during serialization. SupportsShouldProcess provides -WhatIf and -Confirm. The call to ShouldProcess decides whether changes should happen. This lesson previews its entire write operation before creating the directory. UTF8 encoding in Windows PowerShell 5.1 includes a byte-order mark; PowerShell 7 uses a different UTF8 default.

Step by step

1. Open Demo.ps1 in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
.\PowerShell\07-Files\Demo.ps1 -WhatIf
.\PowerShell\07-Files\Demo.ps1
```

Expected result

```
First command previews the write without changing files. Second command: Rows loaded: 2; total: 50
minutes.
```

Exercise

Import the JSON file and print only topics with at least 25 minutes. Run WhatIf before making edits.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Get-Content -Raw returns the complete JSON text.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/powershell-07-json-filter.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/powershell-07-json-filter
# Implement the exercise, then verify:
.\PowerShell\07-Files\Demo.ps1
git diff -- PowerShell/07-Files/Demo.ps1
git add -- PowerShell/07-Files/Demo.ps1
git diff --cached
git diff --cached --check
git commit -m "feat(powershell): filter saved JSON study data"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Keep this commit limited to one exercise; explain the rule and the verification in its message body.

For GitHub practice, publish deliberately with `git push -u origin feat/powershell-07-json-filter`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

Demo.ps1

```
[CmdletBinding(SupportsShouldProcess)]
param()
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

$outputDirectory = Join-Path $PSScriptRoot 'output'
if (-not $PSCmdlet.ShouldProcess($outputDirectory, 'Create sample CSV and JSON')) {
  return
}
New-Item -ItemType Directory -Path $outputDirectory -Force | Out-Null
$csvPath = Join-Path $outputDirectory 'sessions.csv'
$jsonPath = Join-Path $outputDirectory 'sessions.json'
$sessions = @(
  [pscustomobject]@{
    Topic = 'C#'
    Minutes = 30
  }
  [pscustomobject]@{
    Topic = 'PowerShell'
    Minutes = 20
  }
)
$sessions | Export-Csv -LiteralPath $csvPath -NoTypeInformation -Encoding UTF8
$sessions | ConvertTo-Json -Depth 3 | Set-Content -LiteralPath $jsonPath -Encoding UTF8
$loaded = @(Import-Csv -LiteralPath $csvPath)
$total = ($loaded | ForEach-Object {
    [int]$_.Minutes
  } | Measure-Object -Sum).Sum
Write-Output "Rows loaded: $($loaded.Count); total: $total minutes"
```

07-Files exercise solution

Import the JSON file and print only topics with at least 25 minutes. Run WhatIf before making edits.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
Get-Content -LiteralPath $jsonPath -Raw | ConvertFrom-Json |
  Where-Object {
  $_.Minutes -ge 25
```

```
} |
  Select-Object -ExpandProperty Topic
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

08-Errors: Errors and external process exit codes

Level: Advanced within this course | Suggested time: 60-90 minutes

Goal

Learn terminating errors, catch, finally, and native programs. Complete 07-Files first.

Understand the idea

Many cmdlet failures are non-terminating by default. -ErrorAction Stop converts them into terminating errors that try/catch can handle. ErrorActionPreference applies a default within the script scope. Catching a specific exception avoids hiding unrelated failures. finally runs whether the try block succeeds or fails and is suitable for cleanup. Native executables such as dotnet communicate failure through an exit code. Check LASTEXITCODE immediately after the executable. It is different from the success flag $? and is not a general cmdlet error flag. Scripts used in automation should fail clearly instead of continuing with incomplete results.

Step by step

1. Open Demo.ps1 in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
.\PowerShell\08-Errors\Demo.ps1 -Verbose
```

Expected result

```
Expected: the demonstration file does not exist.
An installed .NET SDK version.
The dotnet command succeeded.
Verbose cleanup text is also displayed.
```

Exercise

Call dotnet with an invalid subcommand, capture LASTEXITCODE immediately, and confirm it is nonzero. Handle that expected failure without failing the whole demo.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Run the experiment in a separate terminal or separate script.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/powershell-08-native-exit.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/powershell-08-native-exit
# Implement the exercise, then verify:
.\PowerShell\08-Errors\Demo.ps1
git diff -- PowerShell/08-Errors/Demo.ps1
git add -- PowerShell/08-Errors/Demo.ps1
git diff --cached
git diff --cached --check
git commit -m "test(powershell): check native failure exit codes"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Review the branch's history and include validation evidence in its pull request.

For GitHub practice, publish deliberately with `git push -u origin feat/powershell-08-native-exit`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

Demo.ps1

```
[CmdletBinding()]
param()
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

try {
  Get-Content -LiteralPath (Join-Path $PSScriptRoot 'deliberately-missing.txt') -ErrorAction Stop
} catch [System.Management.Automation.ItemNotFoundException] {
  Write-Output 'Expected: the demonstration file does not exist.'
} finally {
  Write-Verbose 'The finally block runs after success or failure.'
}

# Native tools use process exit codes; ErrorActionPreference alone is insufficient in 5.1.
& dotnet --version
if ($LASTEXITCODE -ne 0) {
  throw "dotnet failed with exit code $LASTEXITCODE"
}
Write-Output 'The dotnet command succeeded.'
```

08-Errors exercise solution

Call dotnet with an invalid subcommand, capture LASTEXITCODE immediately, and confirm it is nonzero. Handle that expected failure without failing the whole demo.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
& dotnet definitely-not-a-real-command
$commandExit = $LASTEXITCODE
if ($commandExit -eq 0) {
  throw 'Expected the invalid command to fail.'
}
Write-Output "Expected failure code: $commandExit"
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

09-Modules: Reusable modules and reporting

Level: Advanced within this course | Suggested time: 60-90 minutes

Goal

Learn module boundaries, exports, begin/process/end, and aggregation. Complete 08-Errors first.

Understand the idea

A .psm1 file contains reusable module code. Import-Module loads it into the current process. Export-ModuleMember controls the functions exposed to callers. A begin block initializes state, process handles each input item, and end produces a final aggregate. The module here validates each object's fields and uses a typed collection to accumulate entries. Functions should document their accepted input shape and return structured results so callers can filter or export them. This small module does not require an online gallery or installation. A larger module can add a .psd1 manifest with version information and declared exports.

Step by step

1. Open Demo.ps1 in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
.\PowerShell\09-Modules\Demo.ps1
```

Expected result

```
C#: 2 sessions, 75 minutes
PowerShell: 1 sessions, 20 minutes
```

Exercise

Add AverageMinutes to each summary object. Check C# gives 37.5.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Compute the average on the numeric Minutes property inside the end block.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/powershell-09-average-minutes.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/powershell-09-average-minutes
# Implement the exercise, then verify:
.\PowerShell\09-Modules\Demo.ps1
git diff -- PowerShell/09-Modules/Demo.ps1 PowerShell/09-Modules/StudyTools.psm1
git add -- PowerShell/09-Modules/Demo.ps1 PowerShell/09-Modules/StudyTools.psm1
git diff --cached
git diff --cached --check
git commit -m "feat(powershell): add average minutes to summaries"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Review the branch's history and include validation evidence in its pull request.

For GitHub practice, publish deliberately with `git push -u origin feat/powershell-09-average-minutes`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

Demo.ps1

```
[CmdletBinding()]
param()
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'
Import-Module (Join-Path $PSScriptRoot 'StudyTools.psm1') -Force

$sessions = @(
  [pscustomobject]@{
    Topic = 'C#'
    Minutes = 30
  }
  [pscustomobject]@{
    Topic = 'C#'
    Minutes = 45
  }
  [pscustomobject]@{
    Topic = 'PowerShell'
    Minutes = 20
  }
)
$sessions | Get-StudySummary | ForEach-Object {
  '{0}: {1} sessions, {2} minutes' -f $_.Topic, $_.Sessions, $_.TotalMinutes
}
```

StudyTools.psm1

```
# Summarize validated study-session objects without formatting their output.
Set-StrictMode -Version Latest
function Get-StudySummary {
  <#
  .SYNOPSIS
  Group sessions by topic and calculate counts and total minutes.
  .DESCRIPTION
  Accept pipeline objects with a nonblank Topic and nonnegative integer Minutes.
  Emit summary objects with Topic, Sessions, and TotalMinutes properties.
  #>
  [CmdletBinding()]
  param(
    [Parameter(Mandatory, ValueFromPipeline)]
    [psobject]$Session
  )
  begin {
    $items = New-Object 'System.Collections.Generic.List[object]'
```

```
  }
  process {
    if ([string]::IsNullOrWhiteSpace([string]$Session.Topic)) {
      throw 'Topic is required.'
    }
    $parsedMinutes = 0
    if (-not [int]::TryParse([string]$Session.Minutes, [ref]$parsedMinutes) -or $parsedMinutes -lt 0) {
      throw 'Minutes must be a nonnegative whole number.'
    }
    $items.Add([pscustomobject]@{
        Topic = [string]$Session.Topic
        Minutes = $parsedMinutes
    })
  }
  end {
    foreach ($group in ($items | Group-Object Topic | Sort-Object Name)) {
      [pscustomobject]@{
        Topic = $group.Name
        Sessions = $group.Count
        TotalMinutes = ($group.Group | Measure-Object -Property Minutes -Sum).Sum
      }
    }
  }
}
Export-ModuleMember -Function Get-StudySummary
```

09-Modules exercise solution

Add AverageMinutes to each summary object. Check C# gives 37.5.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
AverageMinutes = ($group.Group | Measure-Object -Property Minutes -Average).Average
## Add the property to the module's output hashtable.
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

10-Capstone: Control the .NET task API

Level: Advanced within this course | Suggested time: 60-90 minutes

Goal

Learn http requests, json payloads, api contracts, and automation. Complete 09-Modules first.

Understand the idea

Invoke-RestMethod sends HTTP requests and deserializes JSON responses. GET reads; POST creates; PUT updates; DELETE removes in this teaching API. Content-Type tells the server how to interpret the request body. Explicit UTF8 bytes preserve non-ASCII titles across PowerShell versions. The API defines paths and status codes; a failing HTTP status raises an error in this script. ShouldProcess protects mutations and supports WhatIf. Keep formatting at the presentation boundary so returned tasks remain objects. The API must be running before List or a mutation. BaseUri allows a different loopback port. This client and the server are local learning tools without authentication.

Step by step

1. Open Demo.ps1 in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. If the example is a server, leave it running and use a second terminal for requests; stop it with Ctrl+C afterward.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
.\PowerShell\10-Capstone\Demo.ps1
# After starting the server in a second terminal:
.\PowerShell\10-Capstone\Demo.ps1 -Action Add -Title 'Learn PowerShell'
.\PowerShell\10-Capstone\Demo.ps1 -Action List
```

Expected result

```
Default run prints help. With the server running, Add returns an object containing id, title, and
isComplete; List returns saved tasks.
```

Exercise

Preview completion with WhatIf, then complete a real task and verify IsComplete. Use the ID returned by Add.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Capture Add's result in a variable instead of assuming the ID is 1.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/powershell-10-task-client.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/powershell-10-task-client
# Implement the exercise, then verify:
.\PowerShell\10-Capstone\Demo.ps1 -Action List
git diff -- PowerShell/10-Capstone/Demo.ps1
git add -- PowerShell/10-Capstone/Demo.ps1
git diff --cached
git diff --cached --check
git commit -m "test(powershell): verify task client completion"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Review the branch's history and include validation evidence in its pull request.

For GitHub practice, publish deliberately with `git push -u origin feat/powershell-10-task-client`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

Demo.ps1

```
[CmdletBinding(SupportsShouldProcess)]
param(
  [ValidateSet('Help', 'List', 'Add', 'Complete', 'Remove')]
  [string]$Action = 'Help',
  [string]$Title,
  [int]$Id,
  [string]$BaseUri = 'http://127.0.0.1:5080'
)
Set-StrictMode -Version Latest
$ErrorActionPreference = 'Stop'

if ($Action -eq 'Help') {
  Write-Output ('Start DotNet10/10-Capstone, then use -Action List, Add -Title text, ' +
      'Complete -Id n, or Remove -Id n.')
  return
}
$uri = $BaseUri.TrimEnd('/') + '/tasks'
switch ($Action) {
  'List' {
    Invoke-RestMethod -Uri $uri -Method Get
  }
  'Add' {
    if ([string]::IsNullOrWhiteSpace($Title)) {
      throw 'Supply -Title with nonempty text.'
    }
    if ($PSCmdlet.ShouldProcess($uri, 'Add task')) {
      $body = @{
        title = $Title
      } | ConvertTo-Json
      $request = @{
        Uri = $uri
        Method = 'Post'
        ContentType = 'application/json; charset=utf-8'
        Body = [System.Text.Encoding]::UTF8.GetBytes($body)
      }
      Invoke-RestMethod @request
    }
  }
  'Complete' {
    if ($Id -lt 1) {
      throw 'Supply a positive -Id.'
    }
    if ($PSCmdlet.ShouldProcess("$uri/$Id/complete", 'Complete task')) {
      Invoke-RestMethod -Uri "$uri/$Id/complete" -Method Put
    }
  }
```

```
  'Remove' {
    if ($Id -lt 1) {
      throw 'Supply a positive -Id.'
    }
    if ($PSCmdlet.ShouldProcess("$uri/$Id", 'Remove task')) {
      Invoke-RestMethod -Uri "$uri/$Id" -Method Delete
      Write-Output "Removed task $Id"
    }
  }
}
```

10-Capstone exercise solution

Preview completion with WhatIf, then complete a real task and verify IsComplete. Use the ID returned by Add.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
$task = .\PowerShell\10-Capstone\Demo.ps1 -Action Add -Title 'Practice APIs'
.\PowerShell\10-Capstone\Demo.ps1 -Action Complete -Id $task.id -WhatIf
.\PowerShell\10-Capstone\Demo.ps1 -Action Complete -Id $task.id
.\PowerShell\10-Capstone\Demo.ps1 -Action List
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.